CropCare AI - Project Summary

Project Overview: CropCare AI

Based on the provided guidelines, I selected the Plant Disease Detection project from the Agriculture & Intelligent Supply Chains track. The goal is to identify diseases from images of plant leaves, solving a critical real-world problem (crop loss).

I have built this as an end-to-end AI application, fulfilling every single requirement outlined in the KIET guidelines.

1. AI/ML Model Layer (The Brain)
- What it is: A deep learning model built using PyTorch.
- Architecture: I implemented a script to fine-tune MobileNetV2, a highly efficient Convolutional Neural Network (CNN).
- Files Created:
  - train.py: A complete script to load the PlantVillage dataset and train the model from scratch.
  - inference.py: A script that loads the trained weights to make predictions. I also added a "mock mode" to this script so the backend works instantly even before you download the massive dataset and train the model.

2. Backend API Layer (The Server)
- What it is: A robust, high-performance web server built with FastAPI (Python).
- Features: It exposes a /predict endpoint that receives an image from the frontend, sends it to the AI model for analysis, and returns the predicted disease name and a confidence score.
- Files Created: main.py and requirements.txt. FastAPI also automatically generates the required API documentation for you.

3. Frontend UI Layer (The Web App)
- What it is: A visually stunning, modern web interface.
- Design: I used HTML, Vanilla CSS, and JavaScript. I implemented a premium glassmorphism design (frosted glass panels, dynamic animated background blobs, hover effects).
- Features: It includes a drag-and-drop zone for users to upload their leaf images, a loading spinner for analysis, and a clean results card.
- Files Created: index.html, styles.css, and script.js.

4. Technical Documentation (The Deliverables)

CropCare AI - Project Summary

- Technical Report: I wrote TECHNICAL_REPORT.md which includes the strict Engineering Justifications mandated by your PDF. It explains exactly why we chose this problem, dataset, model (MobileNetV2), tech stack, and architecture.
- Title Page: Created a TITLE_PAGE.md matching the KIET Summer Internship format.
- README: Created a README.md with step-by-step instructions on how to install dependencies, train the model, and run the server.

How it all connects:

1. The user opens the web page (index.html) and uploads a leaf image.
2. The JavaScript (script.js) sends that image over the network to the FastAPI backend (main.py).
3. The backend passes the image to the AI script (inference.py).
4. The AI analyzes the image, identifies the disease, and sends the result back to the backend.
5. The backend forwards the result to the frontend, which displays it beautifully to the user.